

ControlFlow.S

```

1  @ =====
2  @
3  @ 作業說明: RP2040 Raspberry Pi Pico Chapter 5 Exercises (5-2, 5-3, 5-4)
4  @ 修正說明: 修正 Cortex-M0+ (Thumb-1) 針對高位暫存器 (R8) 的 Push/Pop/Movs 限制
5  @
6  @ U13157005 吳翊弘
7  @ =====
8
9  .syntax unified
10 .cpu cortex-m0plus
11 .thumb
12 .thumb_func      @ SDK 要求的 Thumb 函式標籤
13 .global main     @ 提供 Linker 程式的進入點 (main)
14
15 main:
16     PUSH {R4, R5, LR}  @ 保存暫存器與返回位址 (僅使用 R0-R7 低位暫存器)
17
18     BL  stdio_init_all @ 初始化 stdio (自動開啟 USB 輸出)
19
20 @ =====
21 @ 練習 5-2: 實作 SELECT / CASE 多分支選擇結構
22 @ =====
23 ex_5_2_select_case:
24     MOVs R0, #2        @ 測試設定: 定義 number = 2
25
26     @ 檢查 CASE 1
27     CMP R0, #1         @ 比較 number 是否等於 1
28     BEQ case_1         @ 若等於 1, 跳轉到 case_1
29
30     @ 檢查 CASE 2
31     CMP R0, #2         @ 比較 number 是否等於 2
32     BEQ case_2         @ 若等於 2, 跳轉到 case_2
33
34     @ CASE ELSE (預設情況)
35     B   case_else      @ 若都不符合, 跳轉到 case_else
36
37 case_1:
38     MOVs R3, #11       @ 測試用: 記錄結果到 R3
39     B   end_select     @ 執行完成後跳出 SELECT 結構
40
41 case_2:
42     MOVs R3, #22       @ 測試用: 記錄結果到 R3
43     B   end_select     @ 執行完成後跳出 SELECT 結構
44
45 case_else:
46     MOVs R3, #99       @ 測試用: 記錄結果到 R3
47     B   end_select     @ 跳出 SELECT 結構
48
49 end_select:
50
51 @ =====

```

```

52 @ 練習 5-3: 實作 DO / WHILE (DO ... UNTIL) 迴圈
53 @ =====
54 ex_5_3_do_while:
55     MOVS R2, #0          @ 初始化計數器 R2 = 0
56
57 do_loop:
58     ADDS R2, #1          @ 迴圈主體內容: R2 = R2 + 1
59
60     @ UNTIL condition (直到 R2 == 5 時結束迴圈)
61     CMP R2, #5           @ 檢查條件: 比較 R2 與 5
62     BLT do_loop          @ 若 R2 < 5 則繼續重複執行
63
64 @ =====
65 @ 練習 5-4: 準備 64 位元整數並轉換成 ASCII
66 @ =====
67 ex_5_4_prepare_data:
68     @ 1. 設定測試資料 (64 位元數值: 0x12345678ABCD9012)
69     LDR R4, =val_high    @ 載入高 32 位元數值 0x12345678 位址
70     LDR R4, [R4]
71     LDR R5, =val_low     @ 載入低 32 位元數值 0xABCD9012 位址
72     LDR R5, [R5]
73
74     @ 2. 轉換低 32 位元 (R5) 並填寫至字串後半部 (8 個字元)
75     LDR R1, =hexstr       @ 取得字串緩衝區起始位址
76     ADDS R1, #17          @ 移動指標到字串尾端 (第 17 個字元位置)
77     MOV R0, R5            @ 將低位 32 位元傳入處理暫存器 R0
78     BL convert_32bit_to_ascii
79
80     @ 3. 轉換高 32 位元 (R4) 並填寫至字串前半部 (8 個字元)
81     LDR R1, =hexstr       @ 重置指標到字串緩衝區起始位址
82     ADDS R1, #9           @ 移動指標到前半段字串尾端
83     MOV R0, R4            @ 將高位 32 位元傳入處理暫存器 R0
84     BL convert_32bit_to_ascii
85
86 @ =====
87 @ 主迴圈 (對應 C 語言的 while(true) { printf(...); sleep_ms(1000); })
88 @ =====
89 main_loop:
90     @ 1. 呼叫 printf("64-bit Register Value = %s\n", hexstr)
91     LDR R0, =printstr     @ 第一個參數 (R0): 格式化字串格式
92     LDR R1, =hexstr       @ 第二個參數 (R1): 已轉好的 64-bit Hex 字串位址
93     BL printf
94
95     @ 2. 呼叫 sleep_ms(1000)
96     LDR R0, =1000         @ 參數 (R0): 1000 毫秒
97     BL sleep_ms
98
99     @ 3. 繼續重複執行迴圈
100    B main_loop
101
102    POP {R4, R5, PC}      @ 返回
103
104 @ =====
105 @ 子程式: convert_32bit_to_ascii

```

```

106 @ 輸入參數:
107 @ R0 - 欲轉換的 32 位元數值
108 @ R1 - 寫入 ASCII 字元的記憶體目標位址
109 @ =====
110 convert_32bit_to_ascii:
111     PUSH {R3, R4, R5, LR} @ 僅使用 Low Registers (R0-R7) 進行 Push/Pop
112     MOVS R5, #8            @ 迴圈計數器 R5 = 8
113
114 conv_loop:
115     MOV R3, R0              @ 複製當前數值到 R3
116     MOVS R4, #0x0F          @ Mask 取最低 4 位元
117     ANDS R3, R4             @ 取出 1 個 Hex 數字 (0x0 ~ 0xF)
118
119     CMP R3, #10             @ 判斷是數字 (0-9) 還是字母 (A-F)
120     BGE conv_letter
121
122 conv_digit:
123     ADDS R3, #'0'           @ 轉為數字 ASCII ('0'-'9')
124     B conv_store
125
126 conv_letter:
127     ADDS R3, #('A'-10)      @ 轉為英文字母 ASCII ('A'-'F')
128
129 conv_store:
130     STRB R3, [R1]           @ 將轉換後的 ASCII 字元存入記憶體 [R1]
131     SUBS R1, #1             @ 目標記憶體位址向左移 1 字元
132     LSRS R0, #4             @ 邏輯右移 4 位元, 準備處理下一個 Hex 碼
133
134     SUBS R5, #1             @ 計數器 -1
135     BNE conv_loop          @ 若尚未轉換完 8 個字元則繼續
136
137     POP {R3, R4, R5, PC}   @ 子程式返回
138
139 .ltorg                     @ 宣告 Literal Pool 常數池, 防止 LDR 溢出
140
141 @ =====
142 @ 資料區段 (Data Section)
143 @ =====
144 .align 4
145 .data
146
147 val_high: .word 0x12345678
148 val_low:  .word 0xABCD9012
149 hexstr:   .asciz "0x0000000000000000"
150 printstr: .asciz " Register Value = %s\n"

```

